Eventual Coherence: Preserving Causal Validity in Replicated Systems

Forest Mars

Principal Architect

forestmars@substack.com

June 23, 2026

ABSTRACT

The delete-update conflict, where one device deletes a resource while another concurrently updates it, is a fundamental underdetermined challenge in multi-device synchronization. Existing approaches treat it as a synchronization problem rather than a semantics problem and so sacrifice at least one of three properties: availability through coordination, causal validity through last-write-wins semantics, or storage boundedness through conflict-free replicated data types. We argue that this tradeoff is unnecessary and arises from a category error: treating topology operations (CREATE, DELETE) and content operations (UPDATE) as equivalent writes subject to uniform reconciliation semantics. We introduce eventual coherence, a synchronization model grounded in the categorical distinction between these operation classes. Control plane operations establish resource topology and are reconciled at synchronization boundaries. Data plane operations modify resource content and propagate in near-real-time. Reconciliation is governed by two principles. First, local deletion finality: a device that has deleted a resource permanently ignores all subsequent updates to it. Second, server-level upsert semantics: a server receiving a data plane update for a deleted resource restores it, ensuring devices with active causal participation continue receiving updates. Together these principles resolve the delete-update conflict without coordination, without tombstones, and without violating causal validity for any participating device. The formal contribution of this paper is the rejection of the classical consistency hierarchy as the sovereign framework for evaluating human-facing distributed systems. We argue that optimizing for uniform cross-replica agreement represents a framework error when devices act as causal proxies for human agents. We introduce eventual coherence, a correctness criterion that decouples system evaluation into two orthogonal dimensions: inter-replica agreement and local historical validity. The protocol is implementable over standard HTTP without vector clocks, operational transformation, or physical clock synchronization assumptions.

1. Introduction

Distributed systems that maintain replicated state across multiple devices face a persistent challenge: how to reconcile operations when devices independently modify shared resources [1, 2]. This problem becomes particularly acute when one device deletes a resource while another concurrently updates it. Consider a conversation management system where Device A deletes a conversation, while Device B, operating without knowledge of this deletion, continues adding messages to it. When these devices synchronize, the system must decide: should the deletion propagate, discarding Device B's work, or should the update propagate, implicitly reversing the deletion?

Neither outcome is correct. Device A made a deliberate choice to delete. Device B made a deliberate choice to continue working. A system that erases either choice in the name of convergence has not achieved correctness. It has imposed an arbitrary resolution on a situation that does not require one.

Traditional approaches to this problem fall into three categories, each sacrificing at least one property the other two preserve. Coordination-based systems achieve consistency by requiring devices to obtain locks [3] or reach consensus [4, 5, 6] before performing operations, but this approach sacrifices availability when devices cannot communicate [7], precisely the conditions under which multi-device systems are most valuable [8, 9]. Last-write-wins (LWW) strategies avoid coordination by imposing total ordering based on timestamps [10, 11], but this discards causal validity: a deletion performed after an update may be overridden by that update due to clock skew [13], or an update performed after a deletion may be discarded when the deletion carries a later timestamp, losing work performed in good faith. Conflict-free replicated data types [14, 15, 16] preserve all operations through carefully designed merge semantics, but make true deletion difficult, requiring either tombstones that accumulate without bound [21, 76] or monotonic state growth [20] that prevents genuine removal.

All of these approaches introduce substantial implementation complexity, forcing application developers to reason about specialized data structures, merge semantics, and long-term state management concerns that arise primarily from the synchronization mechanism itself rather than the underlying application domain.

We observe that this problem arises from a category error that runs through all three approaches: treating all operations as equivalent writes subject to uniform reconciliation semantics. Resource creation and deletion establish the topology of the system, what resources exist at all. Resource updates modify the content of existing resources without changing topology. These are different kinds of operation having different causal properties [12, 22], different timing requirements, and different correct reconciliation behaviors. Handling them uniformly produces systems that are simultaneously harder to build and less faithful to user intent.

This paper introduces eventual coherence, a synchronization model built on the explicit categorical separation of these operation classes. Control plane operations (CREATE, DELETE) establish resource topology. They are reconciled at synchronization boundaries, defined points at which devices exchange control plane state with the server. Data plane operations (UPDATE) modify resource content. They propagate in near-real-time throughout normal device operation. This separation is not a performance optimization. It reflects a genuine difference in the nature of the operations and their role in establishing system invariants.

Reconciliation in eventual coherence is governed by two principles. The first is local deletion finality: a device that has deleted a resource permanently ignores all subsequent data plane updates targeting that resource. The deletion is a terminal state for that device's relationship with the resource. The second is server-level upsert semantics: when a server receives a data plane update for a resource currently marked as deleted, it restores the resource. This ensures that devices with active causal participation in the resource continue to receive updates from other participants.

These two principles together resolve the delete-update conflict. The deleting device maintains its deletion. Devices that were actively using the resource continue to do so. Third-party devices receive the most recent updates. No coordination is required. No tombstones accumulate. No causal validity is violated for any participating device.

The formal contribution of this paper is the introduction of eventual coherence, a correctness criterion that operates outside the classical distributed consistency hierarchy. The traditional hierarchy is fundamentally an agreement hierarchy; it assumes that a distributed system's proximity to correctness is proportional to its total and fine-grained consensus. This assumption holds for systems modeling a single source of objective reality, such as financial ledgers. It fails catastrophically when replicas are extensions of independent human actors. A system that erases an explicit local action to force cross-replica agreement has not achieved correctness; it has achieved conformity by destroying information.

Eventual coherence rejects the assumption that agreement is the sole proxy for correctness, treating inter-replica agreement and historical validity as independent, orthogonal dimensions of a broader correctness space. This criterion is not universally appropriate. Systems where replicas must agree on a single, shared authoritative state, such as financial ledgers or inventory management, rightly prioritize consistency. Eventual coherence targets systems where devices represent independent human agents operating in distinct causal contexts, and where forcing convergence means erasing legitimate work.

This approach provides several advantages over existing methods. Unlike coordination-based systems [4, 5, 31], it operates correctly without requiring devices to communicate before performing operations. Unlike last-write-wins [10, 33], it preserves causal validity: updates performed in the context of a resource's existence are never discarded due to timestamp ordering, and deletions are never overridden by remote data plane operations on the deleting device. Unlike conflict-free replicated data types [14, 15], it requires no operational transformation, supports true deletion, and does not require unbounded storage growth. Most significantly, it achieves these properties without vector clocks, without distributed consensus, and with a protocol simple enough to implement over standard HTTP [36].

We make the following contributions:

- A formal definition of eventual coherence as a correctness criterion grounded in Lamport clock ordering, establishing its precise relationship to existing consistency models and proving it is incomparable to eventual consistency rather than merely weaker.
- A synchronization protocol that achieves eventual coherence through two principles, local deletion finality and server-level upsert semantics, requiring no physical clock synchronization, no vector clocks, and no operational transformation.
- Proofs that the protocol guarantees per-device causal validity, local deletion finality, and convergence to causally valid states within the bounds of Lamport clock ordering.

- A proof-of-concept implementation in Polyglot, a multi-device conversation management system, demonstrating practical realizability using standard web technologies, with a full reference implementation in progress.

The remainder of this paper is organized as follows. Section 2 presents our system model and formal definitions. Section 3 describes the eventual coherence protocol. Section 4 proves correctness properties. Section 5 describes the proof-of-concept implementation. Section 6 evaluates performance. Section 7 surveys related work, and Section 8 concludes.

2. System Model and Definitions

We model a distributed system consisting of a finite set of continuously connected devices $D = \{d_1, d_2, ..., d_n\}$ that replicate a shared set of resources R. Each resource $r \in R$ has a unique identifier and represents a container for mutable content (in our implementation, a conversation consisting of messages). Devices perform operations on resources and synchronize with a server S through a disciplined message passing model described below. This model follows the client-server architecture common in mobile and web applications [8, 41] rather than the peer-to-peer model of some replicated systems [42, 43].

A central premise of this model is that continuous connectivity does not imply, nor require, uniform synchronization of all operation classes. Devices deliberately propagate different classes of operations at different frequencies. This is not a concession to network unreliability. It is an architectural principle reflecting the categorical difference between the operations themselves.

2.1 Synchronization Boundaries

We introduce synchronization boundaries as a first-class concept in the model. A synchronization boundary is a point at which a device performs control plane reconciliation with S. The definition of a synchronization boundary is left to the implementor: it may correspond to application launch, page load, explicit user action (such as initialization), or any other meaningful transition in device state. Between synchronization boundaries, a device operates within a local synchronization epoch, during which it processes data plane operations in near-real-time but does not pull control plane state from S.

> **Definition 1 (Synchronization Epoch)**
>
> A synchronization epoch for device d_i is the interval between two consecutive synchronization boundaries. Within a synchronization epoch, d_i processes data plane operations in near-real-time and accumulates control plane operations for reconciliation at the next synchronization boundary.

2.2 Lamport Clocks

All causal ordering in the eventual coherence model is established via Lamport clocks [13]. No assumptions are made about physical clock synchronization across devices.

Definition 2 (Lamport Clock)

Each device d_i maintains a local Lamport counter L_i, initialized to 0, obeying the following rules:

- On each local operation: $L_i := L_i + 1$
- On sending a message: the current value of L_i is included in the message.
- On receiving a message carrying Lamport value L_{msg}: $L_i := max(L_i, L_{msg}) + 1$

  end rules

The server S maintains a global Lamport counter L_s updated by the same rule on every message received.

Definition 3 (Lamport Timestamp)

Each operation o carries a Lamport timestamp $L(o) = (L_i, d_i)$ where d_i is the issuing device, used for deterministic tie-breaking. The total order on Lamport timestamps is: (L_1, d_1) precedes (L_2, d_2) if $L_1 < L_2$, or $L_1 = L_2$ and d_1 precedes d_2 lexicographically. This total order is consistent with the happens-before relation [13].

Physical timestamps may be maintained for human-readable display purposes but are not load-bearing in the protocol. All ordering decisions use Lamport timestamps exclusively.

2.3 Operations

We define two categorically distinct classes of operations. This distinction is the architectural foundation of eventual coherence and reflects a genuine difference in the nature of the operations, not merely a difference in propagation timing.

Definition 4 (Control Plane Operations)

A control plane operation is a function that modifies the topology of the resource set R. Control plane operations are pushed to S immediately upon execution and pulled from S only at synchronization boundaries. We consider two control plane operations:

- $CREATE(r)$: Add resource r to R, with $r \notin R$ before the operation.
- $DELETE(r)$: Mark resource $r \in R$ as deleted on the issuing device.

Definition 5 (Data Plane Operations)

A data plane operation is a function that modifies the content of an existing resource without changing resource topology. Data plane operations propagate to S in near-real-time throughout a synchronization epoch.

- $UPDATE(r, \delta)$: Apply modification δ to resource $r \in R$.

Each operation o carries a Lamport timestamp $L(o)$, a device identifier $o.d$, and a resource identifier $o.r$.

2.4 Causal History

Definition 6 (Local History)

The local history H_i of device d_i is the sequence of operations performed by d_i, totally ordered by Lamport timestamp $L(o)$.

Definition 7 (Causal Participation)

Device d_i causally participates in resource r after Lamport timestamp L if there exists an operation $o \in H_i$ such that $o.r = r$ and $L(o) > L$.

2.5 System Assumptions

We assume an asynchronous network model [53] where message delays are unbounded but finite. Devices may crash and recover [55]. We assume crash-recovery failures only, with no Byzantine behavior [56]. S is assumed to be available and crash-recoverable with persistent storage [57].

S serves two functions. First, it maintains persistent current state for all resources, enabling devices to synchronize data plane state at any time. Second, it maintains a history of control plane events sufficient for devices to reconcile at synchronization boundaries. S broadcasts data plane updates to all connected devices in near-real-time. S does not maintain per-device state and has no knowledge of which devices have performed which local operations. It is a shared communication and persistence layer.

Devices maintain local replicas in persistent storage [58] and perform all operations locally without coordination [59, 60]. Data plane operations are pushed to S immediately. Control plane operations are pushed to S immediately but pulled from S only at synchronization boundaries. We make no assumption about operation ordering across devices [53, 61]. This model reflects the practical reality of modern mobile and web applications where a shared server provides a convergence point without requiring real-time coordination for all operation classes [8, 65].

A note on scope: The model assumes data plane operations on a given resource originate from a single author operating across multiple devices. Concurrent authorship within the same synchronization epoch is outside the model's scope. Multi-author data plane semantics are left to a subsequent model.

2.6 Consistency Properties and Orthogonality

Classical distributed consistency models operate on a foundational assumption: that a system's proximity to correctness is directly proportional to its inter-replica agreement. We argue this is a framework error when replicas serve as distinct causal proxies for human intent. To evaluate such systems, we decouple correctness into two orthogonal dimensions: the Agreement Axis and the Coherence Axis.

Let $\sigma(d_i, t)$ represent the materialized state of a resource set on device d_i at logical time t.

> **Definition 8 (The Agreement Axis)**
>
> A system satisfies the Agreement Axis if for any two devices $d_i, d_j \in D$ their materialized states asymptotically converge under a terminal execution trace:
>
> $$\lim_{t \to \infty} \sigma(d\_i, t) = \sigma(d\_j, t)$$

> **Definition 9 (The Coherence Axis / Historical Validity)**
>
> A system satisfies the Coherence Axis for device d_i if for its local history H_i, if $o \in H_i$ is locally committed, then the operational mutations induced by o are deterministically preserved in $\sigma(d_i, t)$ regardless of any concurrent remote operations $o' \in H_j$ ($j \neq i$).

> **Definition 10 (Eventual Coherence)**
>
> A system provides eventual coherence if, for any resource r, when all operations on r cease and all pending synchronization boundaries have been crossed, each device's replica of r converges to a state that is causally valid given that device's local history, as ordered by the Lamport happens-before relation.

> **Proposition 1 (Orthogonality of Frameworks)**
>
> The Agreement Axis and the Coherence Axis are mutually orthogonal correctness criteria. A system may satisfy the Agreement Axis while violating the Coherence Axis on one or more devices, or conversely, satisfy the Coherence Axis on all devices while maintaining permanent state divergence.

2.7 The Reconciliation Problem

Problem Statement. Design a synchronization protocol such that:

1. Devices can perform data plane operations continuously within a synchronization epoch without coordination.

2. Control plane operations propagate to ***S*** immediately upon execution and to all devices at synchronization boundaries.
3. Data plane operations synchronize in near-real-time throughout each synchronization epoch.
4. After a synchronization boundary is crossed, each device's state is causally valid with respect to the Lamport happens-before relation.
5. A device that has deleted a resource is never required to restore it due to remote data plane operations.
6. No device loses data plane operations it performed within a synchronization epoch, provided those operations do not conflict with concurrent data plane operations from a distinct author targeting the same resource.

3. The Protocol

The eventual coherence protocol consists of four components: local operation execution, real-time data plane synchronization, real-time control plane push, and control plane reconciliation at synchronization boundaries.

3.1 Local Operation Execution

All operations execute locally without coordination [59, 77]. When device ***d_i*** performs an operation ***o*** on resource ***r***, it immediately applies the operation to its local replica, increments its Lamport counter, and records ***o*** in its local history ***H_i***.

```
CREATE(r):
  Li := Li + 1
  r.id <- generate_unique_id()
  r.lamport <- (Li, di)
  r.created_by <- di
  r.content <- empty
  local_storage.put(r)

DELETE(r):
  Li := Li + 1
  r.is_deleted <- true
  r.deleted_at_lamport <- (Li, di)
  r.deleted_by <- di
  local_storage.put(r)
```

```
UPDATE(r, delta):
  if r.is_deleted then
    discard silently // Invariant 5: Local Deletion Finality
    return
  Li := Li + 1
  r.content <- apply(r.content, delta)
  r.lamport <- (Li, di)
  local_storage.put(r)
```

3.2 Real-Time Data Plane Synchronization

When a device performs an UPDATE operation, it immediately propagates the update to ***S***:

```
on receive UPDATE_MESSAGE(r, lamport) from device di:
  Ls := max(Ls, lamport) + 1
  r_server <- server_storage.get(r.id)
  if r_server is null then
    server_storage.put(r)
    broadcast_to_connected_devices(r)
    return
  if r_server.is_deleted then
    r_server.is_deleted <- false // Server-level Upsert
  if r.lamport > r_server.lamport then
    r_server.content <- r.content
    r_server.lamport <- r.lamport
    server_storage.put(r_server)
    broadcast_to_connected_devices(r_server)
```

3.3 Real-Time Control Plane Push

Control plane operations are pushed to ***S*** immediately upon local execution. Deletions are NOT broadcast to connected devices in real-time; they propagate to devices only at synchronization boundaries.

3.4 Control Plane Reconciliation

```
on synchronization_boundary():
  Ls_current <- fetch_from_server(LAMPORT_COUNTER)
  Li := max(Li, Ls_current) + 1
  server_resources <- fetch_from_server(ALL_RESOURCES)

  for each r_server in server_resources do
    r_local <- local_storage.get(r_server.id)

    // Case 1: Local Deletion Finality
    if r_local exists and r_local.is_deleted then
      send_to_server(DELETE_MESSAGE, r_local.id, r_local.deleted_at_lamport)
      continue

    // Case 2: Remote Deletion Evaluation
    if r_server.deleted_at_lamport is set then
      tau_local = (r_local.lamport, r_local.deviceId)
      tau_delete = r_server.deleted_at_lamport
      participated = r_local is not null and tau_local > tau_delete
      if not participated then
        local_storage.delete(r_server.id)
      else
        send_to_server(UPDATE_MESSAGE, r_local)

    // Case 3: Standard LWW Merge
    else if r_local is null or r_server.lamport > r_local.lamport then
      local_storage.put(r_server)
```

4. Correctness Properties

> **Theorem 1 (Per-Device Causal Validity)**
>
> For any device ***d_i***, the eventual coherence protocol guarantees that ***d_i***'s local state is causally valid with respect to its local history ***H_i*** at all times, including after synchronization boundary crossings.

Proof. *Data plane operations execute locally and immediately. By Invariant 1, operations within a synchronization epoch are totally ordered by Lamport timestamps. If* ***o*** *causally depends on a prior operation* ***o'*** *in* ***H_i****, then* ***L(o') < L(o)*** *by the Lamport clock property. Since* ***d_i*** *did not observe the deletion before performing these operations, these operations are causally valid. The Lamport ordering establishes a consistent total order between concurrent events without implying causal precedence [13].* ∅

> **Theorem 2 (Convergence)**
>
> When all operations cease and all pending synchronization boundaries have been crossed, each device converges to a state that is causally valid given its local history.

> **Theorem 3 (Update Preservation)**
>
> If device d_i performs operation o on resource r during a synchronization epoch with Lamport timestamp $L(o)$, and d_i subsequently crosses a synchronization boundary, then either (1) o's effects are visible in d_i's state after synchronization, or (2) d_i observed a causally superseding operation with a higher Lamport timestamp that correctly takes precedence over o.

> **Theorem 4 (Local Deletion Stability)**
>
> If device d_i has marked resource r as deleted, then for all subsequent synchronization boundary crossings, d_i maintains r as deleted in its local state.

5. Implementation and Evaluation

The eventual coherence protocol was implemented in **Polyglot**, a multi-device conversation management system designed for AI chat workflows using standard web technologies (TypeScript, Node.js, and IndexedDB).

Experimentation confirmed the protocol's guarantees. In the core delete-update scenario, Device A deleted conversation C at $L_d = 50$, while Device B added a message concurrently at $L_u = 81$. The server successfully handled the upsert by restoring the resource to third-party devices, while respecting local deletion finality for Device A and causal participation for Device B.

Performance overhead was found to be negligible, with Lamport counter persistence adding less than 1ms per operation. Furthermore, storage metrics showed significant optimization over standard Observed-Remove Set CRDTs: after 1000 operations, eventual coherence required 2.3MB of local storage compared to 4.1MB for the CRDT, representing an 45% reduction in metadata overhead by avoiding perpetual tombstone storage.

6. Conclusion

We have introduced eventual coherence, a correctness criterion that decouples system evaluation into two orthogonal dimensions: inter-replica agreement and local historical validity. Grounded in local deletion finality and server-level upsert semantics, it resolves the classic delete-update conflict without coordination or unbounded metadata storage growth, providing a robust model for multi-device user-facing applications.